ЛЕКЦИЯ 6. FLUTTER: НАВИГАЦИЯ И РАБОТА С ДАННЫМИ

НАВИГАЦИЯ (NAVIGATOR)

NAVIGATOR 1.0: ОСНОВЫ

```
// Переход на новый экран
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const DetailScreen(),
  ),
);

// Возврат на предыдущий экран
Navigator.pop(context);

// Возврат данных
Navigator.pop(context, 'Результат');

// Получение результата
final result = await Navigator.push<String>(
  context,
  MaterialPageRoute(builder: (context) => const InputScreen())
);
```

```
print('Получено: $result');
```

NAVIGATOR 2.0: ИМЕНОВАННЫЕ МАРШРУТЫ

```
// В MaterialApp
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const HomeScreen(),
    '/detail': (context) => const DetailScreen(),
    '/settings': (context) => const SettingsScreen(),
  },
  onGenerateRoute: (settings) {
    if (settings.name == '/user') {
      final userId = settings.arguments as int;
      return MaterialPageRoute(
        builder: (context) => UserScreen(userId: userId),
      );
    }
    return null;
  },
);
```

```
// Переход
Navigator.pushNamed(context, '/detail');
Navigator.pushNamed(context, '/user', arguments: 42);
```

GOROUTER: СОВРЕМЕННАЯ НАВИГАЦИЯ

```
// pub.dev/packages/go_router
final router = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const HomeScreen(),
    ),
    GoRoute(
      path: '/user/:id',
      builder: (context, state) {
        final id = int.parse(state.pathParameters['id']!);
        return UserScreen(userId: id);
      },
    ),
    ShellRoute(
      builder: (context, state, child) => MainShell(child: child),
      routes: [
        GoRoute(path: '/home', builder: (_, __) => const HomeScreen()),
        GoRoute(path: '/settings', builder: (_, __) => const SettingsScree
      ],
```

```
    ),
  ],
);

// Переход
context.go('/user/42');
context.push('/detail');
```

GOROUTER: ОБРАБОТКА ОШИБОК

```
final router = GoRouter(
  routes: [...],
  errorBuilder: (context, state) => Scaffold(
    body: Center(
      child: Text('Ошибка: ${state.error}'),
    ),
  ),
);

// Редиректы
GoRouter(
  redirect: (context, state) {
    final isLoggedIn = authService.isLoggedIn;
    final isLoginRoute = state.matchedLocation == '/login';

    if (!isLoggedIn && !isLoginRoute) return '/login';
    if (isLoggedIn && isLoginRoute) return '/';
    return null;  // без редиректа
  },
```

```
);
```

УПРАВЛЕНИЕ СОСТОЯНИЕМ

ПРОБЛЕМА СОСТОЯНИЯ

- Данные, которые меняются со временем (счётчик, список, авторизация)
- Необходимо уведомлять UI об изменениях
- Варианты решений:
  - **setState** - простой
  - **Provider** - рекомендуемый
  - **Riverpod** - современный
  - **BLoC/Cubit** - архитектурный
  - **GetX** - альтернативный

SETSTATE: ПРОСТОЙ ПОДХОД

```
class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});

  @override
  State<CounterScreen> createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(child: Text('Счётчик: $_count')),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          setState(() { _count++; });
        },
        child: const Icon(Icons.add),
      ),
```

```
    );
  }
}
```

Подходит для простых случаев. Не масштабируется.

PROVIDER: РЕКОМЕНДУЕМЫЙ ПОДХОД

```
// 1. Определяем ChangeNotifier
class CounterModel extends ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();  // уведомляем слушателей
  }
}

// 2. Подключаем Provider (в корне приложения)
ChangeNotifierProvider(
  create: (_) => CounterModel(),
  child: MyApp(),
)

// 3. Используем
Consumer<CounterModel>(
  builder: (context, counter, child) {
```

```
    return Text('Счётчик: ${counter.count}');
  },
)

// 4. Изменяем
context.read<CounterModel>().increment();
```

PROVIDER: ТИПЫ ПРОВАЙДЕРОВ

Провайдер	Назначение	Пример
Provider	Базовый контейнер	Provider(value: myService)
ChangeNotifierProvider	Для ChangeNotifier	ChangeNotifierProvider(create:)
FutureProvider	Для Future	FutureProvider(create:)
StreamProvider	Для Stream	StreamProvider(create:)
Selector	Оптимизация (поиск)	Selector<Model, int>(...)

RIVERPOD: СОВРЕМЕННЫЙ ПОДХОД

```
// 1. Определяем провайдер
final counterProvider = StateNotifierProvider<CounterNotifier, int>(
  (ref) => CounterNotifier(),
);

class CounterNotifier extends StateNotifier<int> {
  CounterNotifier() : super(0);

  void increment() => state++;
  void decrement() => state--;
}

// 2. Используем
class CounterScreen extends ConsumerWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('Счётчик: $count');
```

```
  }
}

// 3. Изменяем
ref.read(counterProvider.notifier).increment();
```

RIVERPOD: АВТОГЕНЕРАЦИЯ

```
// riverpod_generator (build_runner)
@riverpod
Future<List<User>> users(UsersRef ref) async {
  final response = await http.get(Uri.parse('https://api.example.co
  return (jsonDecode(response.body) as List).map((e) => User.fromJs
}

// Использование
class UserList extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final usersAsync = ref.watch(usersProvider);
    return usersAsync.when(
      loading: () => CircularProgressIndicator(),
      error: (err, stack) => Text('Ошибка: $err'),
      data: (users) => ListView.builder(
        itemCount: users.length,
        itemBuilder: (_, i) => ListTile(title: Text(users[i].name))
      ),
    );
```

```
    }
}
```

BLOC: АРХИТЕКТУРНЫЙ ПОДХОД

```
// Events
abstract class CounterEvent {}
class Increment extends CounterEvent {}
class Decrement extends CounterEvent {}

// Bloc
class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<Increment>((event, emit) => emit(state + 1));
    on<Decrement>((event, emit) => emit(state - 1));
  }
}

// Использование
BlocProvider(
  create: (_) => CounterBloc(),
  child: BlocBuilder<CounterBloc, int>(
    builder: (context, count) {
      return Text('Счётчик: $count');
    },
  ),
)

// Изменение
```

```
context.read<CounterBloc>().add(Increment());
```

СРАВНЕНИЕ ПОДХОДОВ

Подход	Сложность	Тестируемость	Когда использовать
setState	Минимальная	Низкая	Простые виджеты
Provider	Низкая	Средняя	Маленькие-средние проекты
Riverpod	Средняя	Высокая	Современные проекты
BLoC	Высокая	Очень высокая	Сложные архитектуры

СЕТЕВЫЕ ЗАПРОСЫ

HTTP: ПАКЕТ HTTP

```
import 'package:http/http.dart' as http;
import 'dart:convert';

class ApiService {
  final String baseUrl = 'https://jsonplaceholder.typicode.com';

  Future<List<Post>> getPosts() async {
    final response = await http.get(Uri.parse('$baseUrl/posts'));
    if (response.statusCode == 200) {
      final List<dynamic> data = jsonDecode(response.body);
      return data.map((json) => Post.fromJson(json)).toList();
    } else {
      throw Exception('Ошибка загрузки');
    }
  }

  Future<Post> createPost(Post post) async {
    final response = await http.post(
      Uri.parse('$baseUrl/posts'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode(post.toJson()),
    );
```

```
    return Post.fromJson(jsonDecode(response.body));
  }
}
```

DIO: ПРОДВИНУТЫЙ HTTP-КЛИЕНТ

```
import 'package:dio/dio.dart';

final dio = Dio(BaseOptions(
  baseUrl: 'https://api.example.com',
  connectTimeout: const Duration(seconds: 5),
  receiveTimeout: const Duration(seconds: 3),
  headers: {'Authorization': 'Bearer token123'},
));

// Interceptor для логирования
dio.interceptors.add(LogInterceptor(responseBody: true));

// Interceptor для токена
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) async {
    final token = await storage.read(key: 'token');
    options.headers['Authorization'] = 'Bearer $token';
    handler.next(options);
  },
  onError: (error, handler) async {
    if (error.response?.statusCode == 401) {
      final newToken = await refreshToken();
      error.requestOptions.headers['Authorization'] = 'Bearer $newToken';
      final response = await dio.fetch(error.requestOptions);
```

```
      return handler.resolve(response);
    }
    handler.next(error);
  },
));
```

DIO: ОБРАБОТКА ОШИБОК

```
try {
  final response = await dio.get('/data');
} on DioException catch (e) {
  switch (e.type) {
    case DioExceptionType.connectionTimeout:
      print('Тайм-аут подключения');
    case DioExceptionType.receiveTimeout:
      print('Тайм-аут ответа');
    case DioExceptionType.badResponse:
      print('Ошибка сервера: ${e.response?.statusCode}');
      print('Тело: ${e.response?.data}');
    case DioExceptionType.cancel:
      print('Запрос отменён');
    case DioExceptionType.connectionError:
      print('Нет соединения');
    default:
      print('Неизвестная ошибка: ${e.message}');
  }
}
```

МОДЕЛИ ДАННЫХ (JSON)

```
// Ручной парсинг
class User {
  final int id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
    );
  }

  Map<String, dynamic> toJson() => {
    'id': id,
    'name': name,
    'email': email,
```

```
  };
}

// Использование
final user = User.fromJson(responseJson);
```

JSON_SERIALIZABLE: АВТОГЕНЕРАЦИЯ

```
// pubspec.yaml
dependencies:
  json_annotation: ^4.9.0
dev_dependencies:
  build_runner: ^2.4.0
  json_serializable: ^6.8.0

// Модель с аннотациями
@JsonSerializable()
class User {
  final int id;
  final String name;
  final DateTime createdAt;

  User({required this.id, required this.name, required this.created

  factory User.fromJson(Map<String, dynamic> json) => _$UserFromJso
  Map<String, dynamic> toJson() => _$UserToJson(this);
```

```
}

// Генерация: dart run build_runner build
```

ЛОКАЛЬНОЕ ХРАНЕНИЕ ДАННЫХ

SHAREDPREFERENCES

```
import 'package:shared_preferences/shared_preferences.dart';

// Сохранение
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'Иван');
await prefs.setInt('age', 25);
await prefs.setBool('dark_mode', true);
await prefs.setStringList('tags', ['dart', 'flutter']);

// Чтение
final username = prefs.getString('username') ?? 'Гость';
final age = prefs.getInt('age') ?? 0;
final tags = prefs.getStringList('tags') ?? [];

// Удаление
await prefs.remove('username');
await prefs.clear();  // удалить всё
```

SQLITE (SQFLITE)

```
import 'package:sqflite/sqflite.dart';

// Создание БД
final database = openDatabase(
  'my_app.db',
  version: 1,
  onCreate: (db, version) {
    db.execute('''
      CREATE TABLE users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE
      )
    ''');
  },
  onUpgrade: (db, oldVersion, newVersion) {
    if (oldVersion < 2) {
      db.execute('ALTER TABLE users ADD COLUMN age INTEGER DEFAULT 0');
    }
  },
);

// CRUD-операции
await database.insert('users', {'name': 'Иван', 'email': 'ivan@mail.ru'});
```

```
final users = await database.query('users');
await database.update('users', {'name': 'Пётр'}, where: 'id = ?', whereArgs: [1])
await database.delete('users', where: 'id = ?', whereArgs: [1]);
```

HIVE: БЫСТРОЕ ХРАНИЛИЩЕ

```
import 'package:hive/hive.dart';

// Регистрация адаптера (генерируется через build_runner)
@HiveType(typeId: 0)
class User extends HiveObject {
  @HiveField(0)
  late String name;

  @HiveField(1)
  late String email;
}

// Инициализация
await Hive.initFlutter();
Hive.registerAdapter(UserAdapter());

// Открытие бокса
final box = await Hive.openBox<User>('users');

// CRUD
box.add(User()..name = 'Иван'..email = 'ivan@mail.ru');
final users = box.values.toList();
```

```
box.deleteAt(0);
box.clear();
```

СРАВНЕНИЕ ХРАНИЛИЩ

Хранилище	Скорость	Формат	Когда использовать
SharedPreferences	Быстро	Key-Value	Настройки, простые данные
sqflite	Средне	SQL	Сложные запросы, связи
Hive	Очень быстро	Бинарный	Кэш, быстрый доступ
Isar	Очень быстро	Бинарный	NoSQL替代 Hive
ObjectBox	Очень быстро	Бинарный	NoSQL,高速增长

PLATFORM CHANNELS

PLATFORM CHANNELS

- Механизм связи Dart-кода с нативным кодом (Kotlin/Swift)
- Используется, когда нет Flutter-плагина для нужной функции
- Типы:
  - **MethodChannel** - вызов методов
  - **EventChannel** - поток событий
  - **BasicMessageChannel** - простые сообщения

METHODCHANNEL: ПРИМЕР

```
// Dart (lib/main.dart)
const channel = MethodChannel('com.example/battery');

Future<int> getBatteryLevel() async {
  final level = await channel.invokeMethod<int>('getBatteryLevel');
  return level ?? -1;
}

// Слушатель событий от нативного кода
channel.setMethodCallHandler((call) async {
  if (call.method == 'onBatteryChanged') {
    final level = call.arguments as int;
    print('Battery level: $level');
  }
});
```

PLATFORM CHANNEL: НАТИВНАЯ РЕАЛИЗАЦИЯ

```
// Kotlin (android/app/src/main/kotlin/.../MainActivity.kt)
class MainActivity : FlutterActivity() {
    private val CHANNEL = "com.example/battery"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL)
            .setMethodCallHandler { call, result ->
                when (call.method) {
                    "getBatteryLevel" -> {
                        val level = getBatteryLevel()
                        result.success(level)
                    }
                    else -> result.notImplemented()
                }
            }
    }

    private fun getBatteryLevel(): Int {
        val batteryManager = getSystemService(BATTERY_SERVICE) as BatteryManager
        return batteryManager.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPA
    }
```

```
}
```

РАБОТА С РЕСУРСАМИ

ASSETS В FLUTTER

```
# pubspec.yaml
flutter:
  assets:
    - assets/images/
    - assets/data/config.json

# Загрузка изображения
Image.asset('assets/images/logo.png')

# Загрузка JSON
final data = await rootBundle.loadString('assets/data/config.j
final config = jsonDecode(data);

# Загрузка изображения как байты
final data = await rootBundle.load('assets/images/logo.png');
final bytes = data.buffer.asUint8List();
```

ТЕМЫ И СТИЛИ

```
MaterialApp(
  theme: ThemeData(
    colorSchemeSeed: Colors.indigo,
    useMaterial3: true,
    brightness: Brightness.light,
    textTheme: const TextTheme(
      headlineMedium: TextStyle(fontSize: 24, fontWeight: FontWeigh
      bodyLarge: TextStyle(fontSize: 16),
    ),
  ),
  darkTheme: ThemeData(
    colorSchemeSeed: Colors.indigo,
    useMaterial3: true,
    brightness: Brightness.dark,
  ),
  themeMode: ThemeMode.system,  // автоматическое переключение
)

// Использование
```

```
Text('Заголовок', style: Theme.of(context).textTheme.headlineMedium
```

КЭШИРОВАНИЕ ИЗОБРАЖЕНИЙ

```
// pub.dev/packages/cached_network_image
import 'package:cached_network_image/cached_network_image.dart

CachedNetworkImage(
  imageUrl: 'https://example.com/photo.jpg',
  placeholder: (context, url) => CircularProgressIndicator(),
  errorWidget: (context, url, error) => Icon(Icons.error),
  fit: BoxFit.cover,
  width: 200,
  height: 200,
)

// Предзагрузка
await CachedNetworkImage.prefetch('https://example.com/photo.j

// Очистка кэша
await DefaultCacheManager().emptyCache();
```

NAVIGATOR: КАСТОМНЫЕ ПЕРЕХОДЫ

```
// FadeTransition
Navigator.push(
  context,
  PageRouteBuilder(
    pageBuilder: (_, __, ___) => const DetailScreen(),
    transitionsBuilder: (_, animation, __, child) {
      return FadeTransition(
        opacity: CurvedAnimation(
          parent: animation,
          curve: Curves.easeInOut,
        ),
        child: child,
      );
    },
  ),
);

// SlideTransition
Navigator.push(
  context,
```

```
  PageRouteBuilder(
    pageBuilder: (_, __, ___) => const DetailScreen(),
    transitionsBuilder: (_, animation, __, child) {
      final tween = Tween<Offset>(
        begin: const Offset(1.0, 0.0),
        end: Offset.zero,
      ).chain(CurveTween(curve: Curves.easeOut));

      return SlideTransition(
        position: animation.drive(tween),
        child: child,
      );
    },
  ),
);
```

NAVIGATOR: ВЛОЖЕННАЯ НАВИГАЦИЯ

```
// Navigator внутри вкладки
class TabScreen extends StatefulWidget {
  @override
  State<TabScreen> createState() => _TabScreenState();
}

class _TabScreenState extends State<TabScreen> {
  final _navigatorKey = GlobalKey<NavigatorState>();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Navigator(
        key: _navigatorKey,
        onGenerateRoute: (settings) {
          return MaterialPageRoute(
            builder: (_) => const HomeTab(),
          );
        },
      ),
```

```
      bottomNavigationBar: BottomNavigationBar(
        onTap: (index) {
          if (index == 0) {
            _navigatorKey.currentState!.pushNamed('/home');
          }
        },
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Главная
          BottomNavigationBarItem(icon: Icon(Icons.settings), label: 'Нас
        ],
      ),
    );
  }
}
```

GOROUTER: SHELLROUTE

```
// Постоянная нижняя навигация
final router = GoRouter(
  routes: [
    ShellRoute(
      builder: (context, state, child) {
        return ScaffoldWithNavBar(child: child);
      },
      routes: [
        GoRoute(
          path: '/home',
          builder: (_, __) => const HomeScreen(),
        ),
        GoRoute(
          path: '/profile',
          builder: (_, __) => const ProfileScreen(),
        ),
        GoRoute(
          path: '/settings',
          builder: (_, __) => const SettingsScreen(),
        ),
      ],
    ),
  ],
);
```

```
// Scaffold с навбаром
class ScaffoldWithNavBar extends StatelessWidget {
  final Widget child;
  const ScaffoldWithNavBar({required this.child});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: child,
      bottomNavigationBar: NavigationBar(
        selectedIndex: _calculateIndex(context),
        onDestinationSelected: (i) => _onItemTapped(i, context),
        destinations: const [
          NavigationDestination(icon: Icon(Icons.home), label: 'Главная'),
          NavigationDestination(icon: Icon(Icons.person), label: 'Профиль'),
          NavigationDestination(icon: Icon(Icons.settings), label: 'Настройки'),
        ],
      ),
    );
  }
}
```

GOROUTER: TYPED ROUTES

```
// Типизированные маршруты (go_router_builder)
@TypedGoRoute<HomeRoute>(
  path: '/',
  routes: [
    TypedGoRoute<UserRoute>(path: 'user/:id'),
  ],
)
class HomeRoute extends GoRouteData {
  const HomeRoute();

  @override
  Widget build(BuildContext context, GoRouterState state) {
    return const HomeScreen();
  }
}

class UserRoute extends GoRouteData {
  final int id;
  const UserRoute({required this.id});

  @override
  Widget build(BuildContext context, GoRouterState state) {
    return UserScreen(userId: id);
  }
```

```
}

// Переход (типизированный)
context.go(const HomeRoute());
context.go(const UserRoute(id: 42));
```

PROVIDER: MULTIPROVIDER

```
// Комбинирование нескольких провайдеров
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (_) => AuthService()),
    ChangeNotifierProvider(create: (_) => CartModel()),
    ChangeNotifierProvider(create: (_) => ThemeModel()),
    Provider<ApiService>(
      create: (_) => ApiService(),
    ),
  ],
  child: MyApp(),
)

// Доступ к разным провайдерам
class UserScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final auth = context.watch<AuthService>();
    final cart = context.watch<CartModel>();

    return Column(
      children: [
```

```
        Text('Привет, ${auth.currentUser.name}'),
        Text('В корзине: ${cart.items.length} товаров'),
      ],
    );
  }
}
```

PROVIDER: PROXYPROVIDER

```
// Зависимости между провайдерами
MultiProvider(
  providers: [
    Provider<DatabaseService>(create: (_) => DatabaseService()),
    ProxyProvider<DatabaseService, UserService>(
      update: (_, db, __) => UserService(db),
    ),
    ProxyProvider2<UserService, AuthService, OrderService>(
      update: (_, users, auth, __) => OrderService(users, auth),
    ),
  ],
)

// C自动 обновлением
class OrderScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // OrderService пересоздаётся при изменении UserService или AuthServic
    final orderService = context.watch<OrderService>();
    return ListView(
      children: orderService.orders.map((o) =>
        ListTile(title: Text(o.id))
```

```
      ).toList(),
    );
  }
}
```

RIVERPOD: FAMILY MODIFIER

```
// Параметризованные провайдеры
final userProvider = FutureProvider.family<User, int>((ref, userId) async
  final response = await http.get(
    Uri.parse('https://api.example.com/users/$userId'),
  );
  return User.fromJson(jsonDecode(response.body));
});

// Использование с параметром
class UserScreen extends ConsumerWidget {
  final int userId;
  const UserScreen({required this.userId});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final userAsync = ref.watch(userProvider(userId));

    return userAsync.when(
      loading: () => CircularProgressIndicator(),
      error: (err, _) => Text('Ошибка: $err'),
      data: (user) => Text(user.name),
    );
```

```
  }
}

// Поиск с debounce
final searchProvider = FutureProvider.family<List<Product>, String>(
  (ref, query) async {
    if (query.isEmpty) return [];
    ref.onDispose(() {}); // debounce
    final results = await searchProducts(query);
    return results;
  },
);
```

RIVERPOD: AUTODISPOSE

```
// Автоматическая очистка
final userProvider = FutureProvider.autoDispose<User>((ref) async {
  // При уничтожении виджета провайдер очищается
  ref.onDispose(() {
    print('Провайдер очищен');
  });

  final response = await http.get(
    Uri.parse('https://api.example.com/user'),
  );
  return User.fromJson(jsonDecode(response.body));
});

// С тайм-аутом
final profileProvider =
    FutureProvider.autoDispose<Profile>((ref) async {
  // Отмена запроса через 30 секунд
  final cancelToken = CancelToken();
  ref.onDispose(() => cancelToken.cancel());

  final response = await dio.get(
    '/profile',
```

```
    cancelToken: cancelToken,
  );
  return Profile.fromJson(response.data);
});

// С кэшированием (keepAlive)
final configProvider =
    FutureProvider.autoDispose<Config>((ref) async {
  ref.keepAlive(); // не очищать автоматически
  final response = await http.get(Uri.parse('/config'));
  return Config.fromJson(jsonDecode(response.body));
});
```

BLOC: MULTIBLOCPROVIDER

```
// Предоставление нескольких BLoC
MultiBlocProvider(
  providers: [
    BlocProvider<AuthBloc>(create: (_) => AuthBloc()),
    BlocProvider<CartBloc>(create: (_) => CartBloc()),
    BlocProvider<ThemeBloc>(
      create: (_) => ThemeBloc(),
    ),
  ],
  child: MyApp(),
)

// Доступ к разным BLoC
class MainScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        BlocBuilder<AuthBloc, AuthState>(
          builder: (context, state) {
            if (state is Authenticated) {
              return Text('Привет, ${state.user.name}');
            }
            return Text('Гость');
```

```
          },
        ),
        BlocBuilder<CartBloc, CartState>(
          builder: (context, state) {
            return Text('Товаров: ${state.items.length}');
          },
        ),
      ],
    );
  }
}
```

BLOC: BLOCLISTENER

```
// Побочные эффекты (навигация, snackbar)
BlocListener<AuthBloc, AuthState>(
  listener: (context, state) {
    if (state is AuthError) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(state.message)),
      );
    }
    if (state is Authenticated) {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (_) => HomeScreen()),
      );
    }
  },
  child: LoginForm(),
)

// ComposedBlocListener + BlocBuilder
BlocConsumer<OrderBloc, OrderState>(
  listener: (context, state) {
    if (state is OrderSuccess) {
      Navigator.push(context, MaterialPageRoute(
        builder: (_) => ConfirmationScreen(),
```

```
      ));
    }
    if (state is OrderError) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(state.message)),
      );
    }
  },
  builder: (context, state) {
    if (state is OrderLoading) {
      return CircularProgressIndicator();
    }
    return OrderForm();
  },
)
```

NETWORKING: GRAPHQL

```
// pub.dev/packages/graphql_flutter
import 'package:graphql_flutter/graphql_flutter.dart';

// Настройка клиента
final httpLink = HttpLink('https://api.example.com/graphql');
final authLink = AuthLink(
  getToken: () async => 'Bearer $token',
);
final link = authLink.concat(httpLink);

final client = GraphQLClient(
  link: link,
  cache: GraphQLCache(store: InMemoryStore()),
);

// Запрос
String readRepositories = '''
  query ReadRepositories(\$first: Int) {
    repositories(first: \$first) {
      nodes {
        name
        description
      }
    }
```

```
  }
''';

Query(
  options: QueryOptions(
    document: gql(readRepositories),
    variables: {'first': 10},
  ),
  builder: (result, {fetchMore, refetch}) {
    if (result.isLoading) return CircularProgressIndicator();
    if (result.hasException) return Text('Ошибка');
    final repos = result.data?['repositories']['nodes'];
    return ListView.builder(
      itemCount: repos.length,
      itemBuilder: (_, i) => ListTile(title: Text(repos[i]['name'])),
    );
  },
)
```

NETWORKING: WEBSOCKET

```
// pub.dev/packages/web_socket_channel
import 'package:web_socket_channel/web_socket_channel.dart';

// Подключение
final channel = WebSocketChannel.connect(
  Uri.parse('wss://echo.websocket.org'),
);

// Отправка и получение
channel.sink.add('Привет, сервер!');
channel.stream.listen(
  (message) => print('Ответ: $message'),
  onDone: () => print('Соединение закрыто'),
  onError: (error) => print('Ошибка: $error'),
);

// Закрытие
channel.sink.close();

// Чат-приложение (пример)
class ChatScreen extends StatefulWidget {
  @override
  State<ChatScreen> createState() => _ChatScreenState();
}
```

```
class _ChatScreenState extends State<ChatScreen> {
  late WebSocketChannel channel;
  final messages = <String>[];

  @override
  void initState() {
    super.initState();
    channel = WebSocketChannel.connect(
      Uri.parse('wss://chat.example.com/ws'),
    );
    channel.stream.listen((data) {
      setState(() => messages.add(data.toString()));
    });
  }

  @override
  void dispose() {
    channel.sink.close();
    super.dispose();
  }
}
```

NETWORKING: RETROFIT

```
// pub.dev/packages/retrofit
import 'package:retrofit/http.dart';
import 'package:dio/dio.dart';

part 'api_client.g.dart';

@RestApi(baseUrl: 'https://api.example.com')
abstract class ApiClient {
  factory ApiClient(Dio dio, {String baseUrl}) = _ApiClient;

  @GET('/users')
  Future<List<User>> getUsers();

  @GET('/users/{id}')
  Future<User> getUser(@Path('id') int id);

  @POST('/users')
  Future<User> createUser(@Body() User user);

  @PUT('/users/{id}')
  Future<User> updateUser(@Path('id') int id, @Body() User user);

  @DELETE('/users/{id}')
  Future<void> deleteUser(@Path('id') int id);
```

```
}

// Использование
final dio = Dio(BaseOptions(baseUrl: 'https://api.example.com'));
final client = ApiClient(dio);

final users = await client.getUsers();
final user = await client.getUser(42);
```

LOCAL STORAGE: DRIFT

```
// pub.dev/packages/drift (formerly moor)
import 'package:drift/drift.dart';

part 'database.g.dart';

class Users extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get name => text().withLength(min: 1, max: 50)();
  TextColumn get email => text().unique()();
}

@DriftDatabase(tables: [Users])
class AppDatabase extends _$AppDatabase {
  AppDatabase() : super(_openConnection());

  @override
  int get schemaVersion => 1;

  Future<int> insertUser(UsersCompanion entry) {
    return into(users).insert(entry);
  }

  Future<List<User>> getAllUsers() {
    return select(users).get();
```

```
  }

  Stream<List<User>> watchAllUsers() {
    return select(users).watch();
  }

  Future<bool> updateUser(UsersCompanion entry) {
    return update(users).replace(entry);
  }

  Future<int> deleteUser(int id) {
    return (delete(users)..where((t) => t.id.equals(id))).go();
  }
}
```

LOCAL STORAGE: ISAR

```
// pub.dev/packages/isar
import 'package:isar/isar.dart';

part 'user.g.dart';

@collection
class User {
  Id id = Isar.autoIncrement;
  late String name;
  late String email;
  late DateTime createdAt;
}

// Инициализация
final isar = await Isar.open([UserSchema]);

// Запись
await isar.writeTxn(() async {
  final user = User()
    ..name = 'Иван'
    ..email = 'ivan@mail.ru'
    ..createdAt = DateTime.now();
  await isar.users.put(user);
});
```

```
// Запросы
final users = await isar.users
    .where()
    .nameEqualTo('Иван')
    .findAll();

// Поток обновлений
isar.users.where().watch().listen((users) {
  print('Обновлено: ${users.length} пользователей');
});

// Индексы и фильтрация
final sortedUsers = await isar.users
    .where()
    .sortByCreatedAtDesc()
    .findAll();
```

CACHING: DIO_CACHE_INTERCEPTOR

```
// pub.dev/packages/dio_cache_interceptor
import 'package:dio_cache_interceptor/dio_cache_interceptor.dart';

final dio = Dio();

// Настройка кэширования
final cacheOptions = CacheOptions(
  store: MemCacheStore(maxSize: 1024 * 1024, maxEntrySize: 50 * 1024),
  policy: CachePolicy.forceCache,
  maxStale: const Duration(days: 7),
  hitCacheOnErrorExcept: [401, 403],
);

dio.interceptors.add(DioCacheInterceptor(options: cacheOptions));

// Использование
Future<Response> getData() async {
  return dio.get(
    'https://api.example.com/data',
    options: Options(
      extra: {
        'cache': true, // кэшировать ответ
      },
    ),
```

```
  );
}

// Очистка кэша
await cacheOptions.store?.clean();

// Разные политики
// CachePolicy.forceCache — всегда использовать кэш
// CachePolicy.forceCacheOnError — кэш при ошибке
// CachePolicy.noCache — не кэшировать
// CachePolicy.refresh — обновить кэш
```

FORMS: FLUTTER FORM

```
class LoginForm extends StatefulWidget {
  @override
  State<LoginForm> createState() => _LoginFormState();
}

class _LoginFormState extends State<LoginForm> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        children: [
          TextFormField(
            controller: _emailController,
            validator: (value) {
              if (value == null || value.isEmpty) return 'Введите email';
              if (!value.contains('@')) return 'Некорректный email';
              return null;
            },
            decoration: InputDecoration(labelText: 'Email'),
```

```
          ),
          TextFormField(
            controller: _passwordController,
            obscureText: true,
            validator: (value) {
              if (value == null || value.length < 6) {
                return 'Минимум 6 символов';
              }
              return null;
            },
            decoration: InputDecoration(labelText: 'Пароль'),
          ),
          ElevatedButton(
            onPressed: () {
              if (_formKey.currentState!.validate()) {
                _login();
              }
            },
            child: Text('Войти'),
          ),
        ],
      ),
    );
  }
}
```

FORMS: INPUT DECORATION

```
TextFormField(
  decoration: InputDecoration(
    labelText: 'Email',
    hintText: 'example@mail.ru',
    prefixIcon: Icon(Icons.email),
    suffixIcon: IconButton(
      icon: Icon(Icons.clear),
      onPressed: () => _controller.clear(),
    ),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    enabledBorder: OutlineInputBorder(
      borderSide: BorderSide(color: Colors.grey),
    ),
    focusedBorder: OutlineInputBorder(
      borderSide: BorderSide(color: Colors.blue, width: 2),
    ),
    errorBorder: OutlineInputBorder(
      borderSide: BorderSide(color: Colors.red),
    ),
    filled: true,
    fillColor: Colors.grey[100],
    contentPadding: EdgeInsets.symmetric(
```

```
      horizontal: 16, vertical: 12,
    ),
  ),
)

// Label + floating label
InputDecoration(
  labelText: 'Имя',
  labelStyle: TextStyle(color: Colors.grey),
  floatingLabelStyle: TextStyle(color: Colors.blue),
)
```

ERROR HANDLING: RESULT TYPES

```
// pub.dev/packages/fpdart
import 'package:fpdart/fpdart.dart';

// Either для обработки ошибок
TaskEither<String, User> fetchUser(int id) {
  return TaskEither.tryCatch(
    () async {
      final response = await http.get(
        Uri.parse('https://api.example.com/users/$id'),
      );
      if (response.statusCode == 200) {
        return User.fromJson(jsonDecode(response.body));
      }
      throw Exception('Ошибка сервера');
    },
    (error, stackTrace) => 'Ошибка загрузки: $error',
  );
}

// Использование
final result = await fetchUser(42).run();
result.match(
  (error) => print('Ошибка: $error'),
  (user) => print('Пользователь: ${user.name}'),
```

```
);

// Цепочки операций
TaskEither<String, User> updateUser(int id, String name) {
  return fetchUser(id)
      .map((user) => user.copyWith(name: name))
      .flatMap((user) => saveUser(user));
}
```

SERIALIZATION: FREEZED MODELS

```
// pub.dev/packages/freezed
import 'package:freezed_annotation/freezed_annotation.dart';

part 'user.freezed.dart';
part 'user.g.dart';

@freezed
class User with _$User {
  const factory User({
    required int id,
    required String name,
    required String email,
    @Default(0) int age,
  }) = _User;

  factory User.fromJson(Map<String, dynamic> json) =>
      _$UserFromJson(json);
}

// Иммутабельные данные
final user1 = User(id: 1, name: 'Иван', email: 'ivan@mail.ru');
final user2 = user1.copyWith(name: 'Пётр'); // новый экземпляр

// Сравнение
```

```
print(user1 == user2); // false (разные имена)

// Pattern matching
final greeting = switch (user) {
  User(age: < 18) => 'Несовершеннолетний',
  User(age: >= 18) => 'Совершеннолетний',
};

// Union types
@freezed
sealed class AuthState with _$AuthState {
  const factory AuthState.idle() = AuthIdle;
  const factory AuthState.loading() = AuthLoading;
  const factory AuthState.authenticated(User user) = AuthAuthenticated;
  const factory AuthState.error(String message) = AuthError;
}
```

SERIALIZATION: EQUATABLE

```
// pub.dev/packages/equatable
import 'package:equatable/equatable.dart';

class User extends Equatable {
  final int id;
  final String name;
  final String email;

  const User({
    required this.id,
    required this.name,
    required this.email,
  });

  @override
  List<Object?> get props => [id, name, email];
}

// Без Equatable
final user1 = User(id: 1, name: 'Иван', email: 'ivan@mail.ru');
```

```
final user2 = User(id: 1, name: 'Иван', email: 'ivan@mail.ru');
print(user1 == user2); // false (сравнение ссылок)

// С Equatable
print(user1 == user2); // true (сравнение полей)

// Важно для BLoC/Cubit
class UserCubit extends Cubit<User> {
  UserCubit() : super(const User(id: 0, name: '', email: ''));

  void updateName(String name) {
    emit(state.copyWith(name: name)); // без Equatable не сработает
  }
}
```

IMAGE HANDLING: IMAGE_PICKER

```
// pub.dev/packages/image_picker
import 'package:image_picker/image_picker.dart';

class ImagePickerScreen extends StatefulWidget {
  @override
  State<ImagePickerScreen> createState() => _ImagePickerScreenState();
}

class _ImagePickerScreenState extends State<ImagePickerScreen> {
  XFile? _image;

  Future<void> _pickImage(ImageSource source) async {
    final picker = ImagePicker();
    final pickedFile = await picker.pickImage(
      source: source,
      maxWidth: 1800,
      maxHeight: 1800,
      imageQuality: 85,
    );

    if (pickedFile != null) {
      setState(() => _image = pickedFile);
    }
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        if (_image != null)
          Image.file(File(_image!.path), height: 200),

        ElevatedButton.icon(
          onPressed: () => _pickImage(ImageSource.camera),
          icon: Icon(Icons.camera_alt),
          label: Text('Камера'),
        ),
        ElevatedButton.icon(
          onPressed: () => _pickImage(ImageSource.gallery),
          icon: Icon(Icons.photo_library),
          label: Text('Галерея'),
        ),
      ],
    );
  }
}
```

ИТОГИ ЛЕКЦИИ 6

НАВИГАЦИЯ

- Navigator 1.0 и 2.0
- GoRouter (рекомендуемый)
- Именованные маршруты

СОСТОЯНИЕ

- setState (простой)
- Provider / Riverpod
- BLoC

СЕТЬ

- http (базовый)
- Dio (продвинутый)
- JSON модели

ХРАНЕНИЕ

- SharedPreferences
- SQLite (sqflite)
- Hive